



Tema 2.- Diseño lógico de Bases de Datos.

Contenido

Tema 2.-	1
Diseño lógico de Bases de Datos.	1
I. Modelo Entidad - Relación	3
1.- Introducción	3
2.- Ejemplo de partida	4
3.- Las Entidades del ME/R.	4
3.1.- ENTIDAD	4
Representación gráfica	4
3.2.- Atributo	5
Representación de la clave principal	5
Representaciones del Ejemplo	6
3.3.- Dominio	6
4.- Las Relaciones del ME/R	6
4.1.- Relaciones	6
Representación de Relaciones	7
GRADO de una Relación	7
Relaciones REFLEXIVAS	7
2 Entidades + de 1 relación	8
Todas las Relaciones del ejemplo	8
4.2.- Atributos de una Relación	9
4.3.- Tipo de Relación	9
Clases de Relaciones	9
Ejemplo de Clases de Relaciones	10
5.- Modelo Extendido	10
5.1.- Cardinalidad máxima y mínima	10
Ejemplo	11
5.2.- Entidades DÉBILES	11
Representación Entidades Débiles	12
Dependencia de EXISTENCIA	12
Dependencia de IDENTIFICACIÓN	12
6.- Relaciones de grado mayor que 2	13
7.- Generalizaciones	13



II. Modelo Relacional	15
1.- ORIGEN Y OBJETIVOS	15
2.- ESTRUCTURA DEL MODELO RELACIONAL	15
3.- RESTRICCIONES	16
3.1.- Restricciones <i>INHERENTES</i>	16
3.2.- Restricciones de Usuario (<i>SEMÁNTICAS</i>)	17
(a) Restricción de Dominio:	17
(b) Restricción de Clave:	17
(c) Integridad Referencial:	17
(d) Otras restricciones:	18
4.- TRANSFORMACIÓN DEL M E/R AL RELACIONAL.	19
4.1.- Entidades	19
4.2.- Relaciones 1:1	19
4.3.- Relaciones 1:N	20
4.4.- Relaciones M:N	20
4.5.- Relaciones Ternarias	20
5.- ALGEBRA RELACIONAL	21
5.1.- Operaciones <i>UNARIAS</i>	22
1) Selección (<i>select</i>)	22
2) Proyección	22
5.2.- Operaciones <i>BINARIAS</i>	22
1) Unión ($R \cup T$)	23
2) Diferencia ($R - T$)	23
3) Producto Cartesiano ($R \times T$)	23
5.3.- OPERACIONES DERIVADAS.	24
1) Intersección ($R \cap T$)	24
2) División (R/T)	24
3) Reunión ($R * T$)	24
6.- LENGUAJES RELACIONALES	25
III. Normalización	26
1.- TEORÍA DE LA NORMALIZACIÓN	26
2.- NOCIÓN INTUITIVA DE LAS FORMAS NORMALES	29
3.- DEPENDENCIAS FUNCIONALES	32
3.1.- Dependencia funcional plena o completa	34
3.2.- Dependencia funcional transitiva	35
4.- DEFINICIÓN FORMAL DE LAS TRES PRIMERAS FORMAS NORMALES	37
5.- DESCOMPOSICIÓN DE RELACIONES	39
a.- Descomposición sin pérdida de información	40
b.- Descomposición sin pérdida de dependencia funcional	41
c.- Descomposición en proyecciones independientes	42
6.- CONSIDERACIONES FINALES SOBRE LA TEORÍA DE LA NORMALIZACIÓN	43



I. Modelo Entidad - Relación

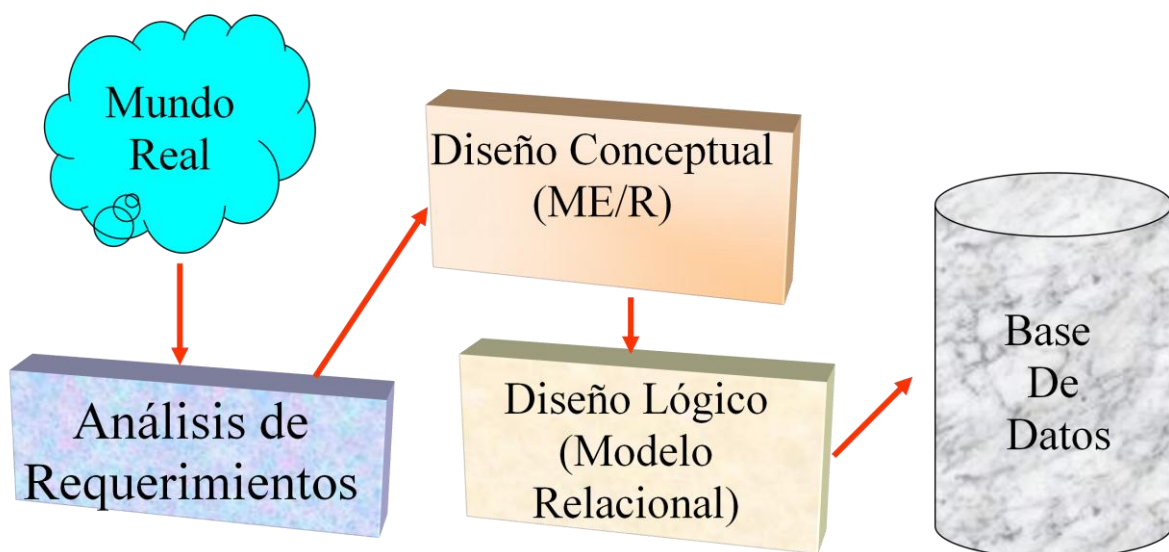
1.- Introducción

El modelo Entidad / Relación (ME/R), propuesto por Peter P. Chen en sus artículos, se basa en **entidades** (cualquier objeto de interés para el universo descrito) *que se relacionan o asocian entre sí*.

El ME/R es un modelo de alto nivel, que *nos permitirá representar el mundo que queremos con un lenguaje*, una estructura más cercana a nosotros.

El ME/R lo utilizaremos dentro del proceso de creación de BD:

1. A partir de la realidad, estudiarla (investigando, entrevistando los usuarios,...) haciendo el **ANÁLISIS de requerimientos** (¿Qué se quiere?). El resultado será un conjunto de requerimientos redactados de forma concisa.
2. A partir del Análisis de requerimientos, diseñar el **Esquema CONCEPTUAL** de la BD con un modelo de alto nivel (ME/R).
3. A partir del Esquema conceptual, **implementar la BD** en un SGBD comercial (Relacional), obteniendo el esquema lógico.





2.- Ejemplo de partida

Se trata de una compañía, y el resultado del análisis de requerimientos es el siguiente:

- La compañía está organizada en departamentos. Cada uno tiene nombre único, número único y un empleado que la dirige. Nos interesa la fecha que empezó a dirigirlo.
- Cada departamento controla una serie de **proyectos**. Cada uno tiene nombre y número únicos.
- De cada **empleado** nos interesa el nombre (nombre y apellidos), DNI, dirección, teléfono, sueldo y fecha de nacimiento. Todo empleado está asignado a un deptº y tendrá un supervisor. Puede trabajar en más de un Proyecto y trabajará un determinado número de horas a la semana en cada proyecto.
- Queremos saber también los **familiares** de cada empleado, para administrar los temas de seguros. Queremos saber el nombre, fecha de nacimiento y parentesco con el empleado.

3.- Las Entidades del ME/R.

3.1.- ENTIDAD

Def.: "Una persona, lugar, cosa, concepto o suceso, real o abstracto, de interés para la empresa"

Por ejemplo, Los Empleados son Entidades.

Llamaremos **TIPO DE ENTIDAD** a la estructura genérica (*Empleado*) y **OCURRENCIA** de entidad a cada una de las realizaciones concretas de ese tipo de entidad (*Antonio Rico*).

Representación gráfica

Representaremos un *tipo de entidad* mediante un rectángulo etiquetado con el nombre del tipo de entidad:



Departamento

Proyecto

Empleado

Familiar

3.2.- Atributo

Es cada una de las **propiedades o características que tiene un tipo de entidad o de relación**.

Por ejemplo, en la entidad **Empleado** tendremos los atributos *nombre, DNI, dirección, teléfono, sueldo y fecha de nacimiento*.

Una **Ocurrencia** de la entidad tendrá un "valor" para cada atributo, por ejemplo, *Antonio Rico, 19.999.335, 925-22.33.45, 150.000pts y 15-6-90*.

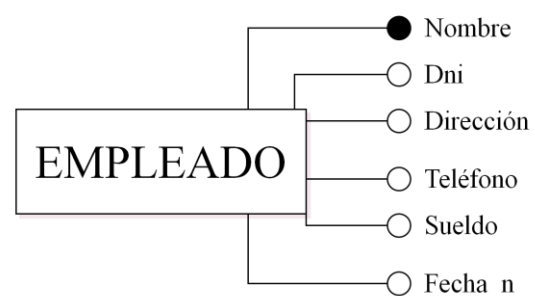
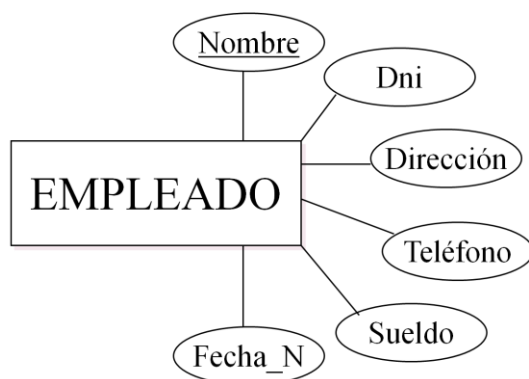
Para poder identificar cada ocurrencia de una entidad necesitamos que algún atributo (o conjunto de atributos) les represente unívocamente.

De los atributos que cumplen la condición anterior, triaríamos uno que denominaríamos **Clave Principal**. Y el resto serían **Claves candidatas**.

Imp.: "**Todas la entidades han de tener una clave principal**" (Es una de las restricciones del ME/R)

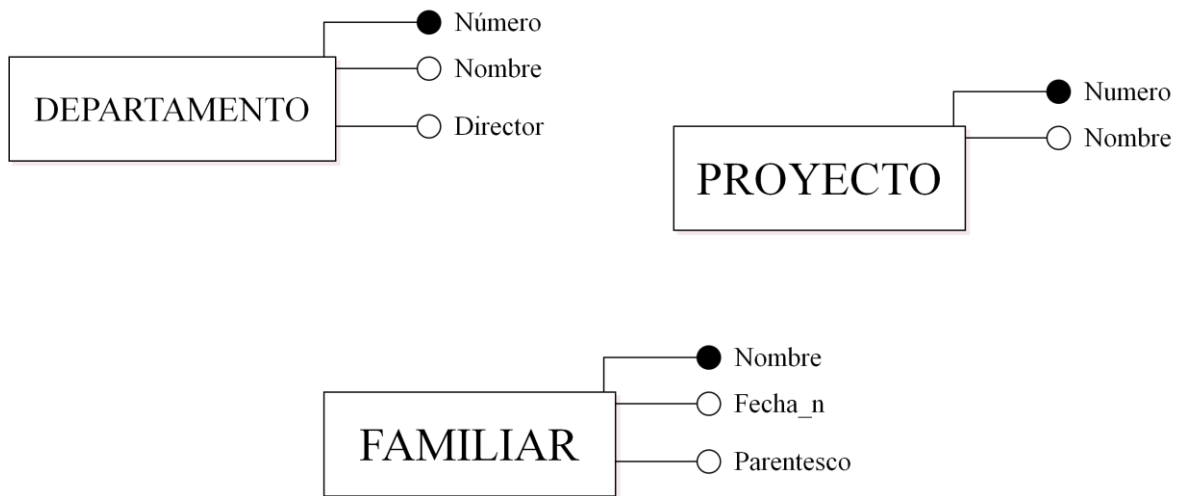
Representación de la clave principal

Puede ser de dos formas:





Representaciones del Ejemplo



3.3.- Dominio

Es el conjunto de posibles valores que puede tomar un atributo.

P.e.: El dominio del atributo DNI seria los números enteros de 8 cifras.

Más de un atributo puede compartir el mismo dominio. Por ejemplo, si incluimos el atributo fecha de incorporación a la compañía en la entidad EMPLEADO, tiene el mismo dominio que fecha_n.

4.- Las Relaciones del ME/R

4.1.- Relaciones

Es una asociación o correspondencia entre entidades.

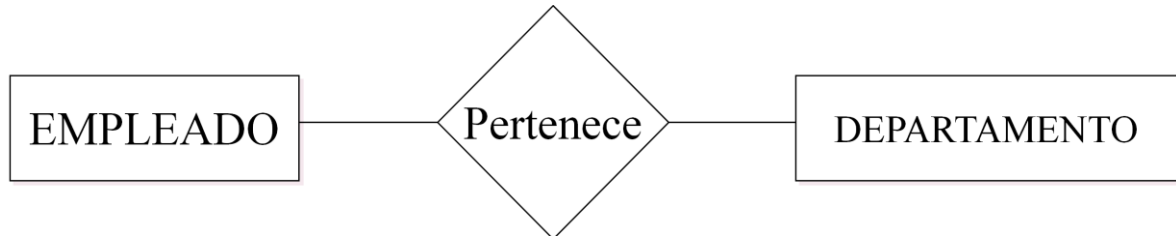
El **Tipo de Relación** será la estructura genérica del conjunto de relaciones existente entre dos o más tipos de entidad.

La **Ocurrencia de Relación** será la vinculación existente entre las ocurrencias concretas de cada uno de los tipos de entidad que intervienen en la Relación.



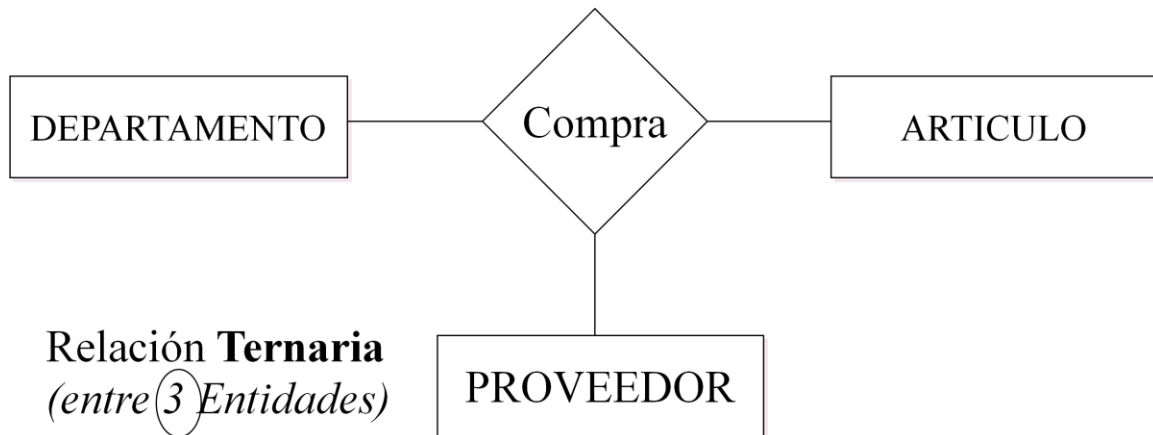
Representación de Relaciones

La Relación se representa por un rombo, con el nombre de la relación en su interior. Uniremos el rombo con ambas Entidades por medio de líneas.



Relación **Binaria** (*intervienen dos entidades*)

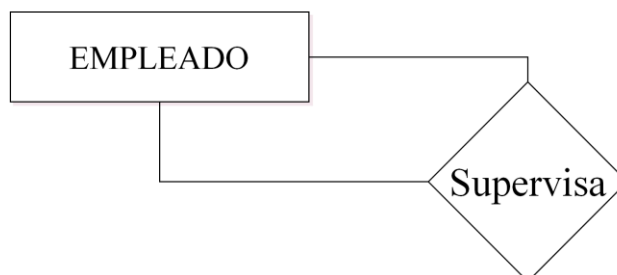
GRADO de una Relación



Relación **Ternaria**
(entre 3 Entidades)

Representa el **GRADO** de una Relación

Relaciones REFLEXIVAS

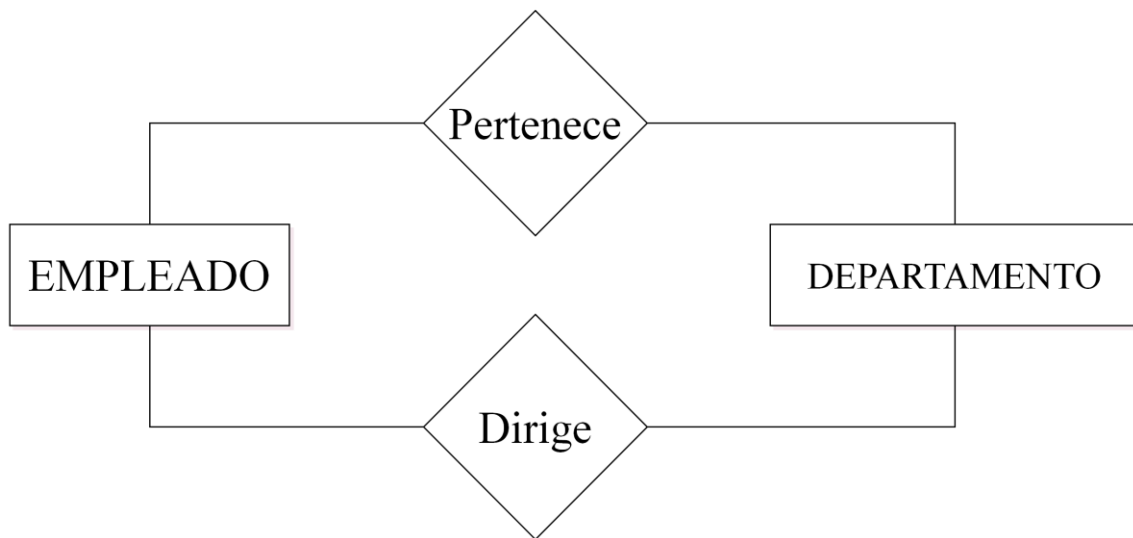


Cuando interviene sólo una Entidad la relación es **REFLEXIVA**.

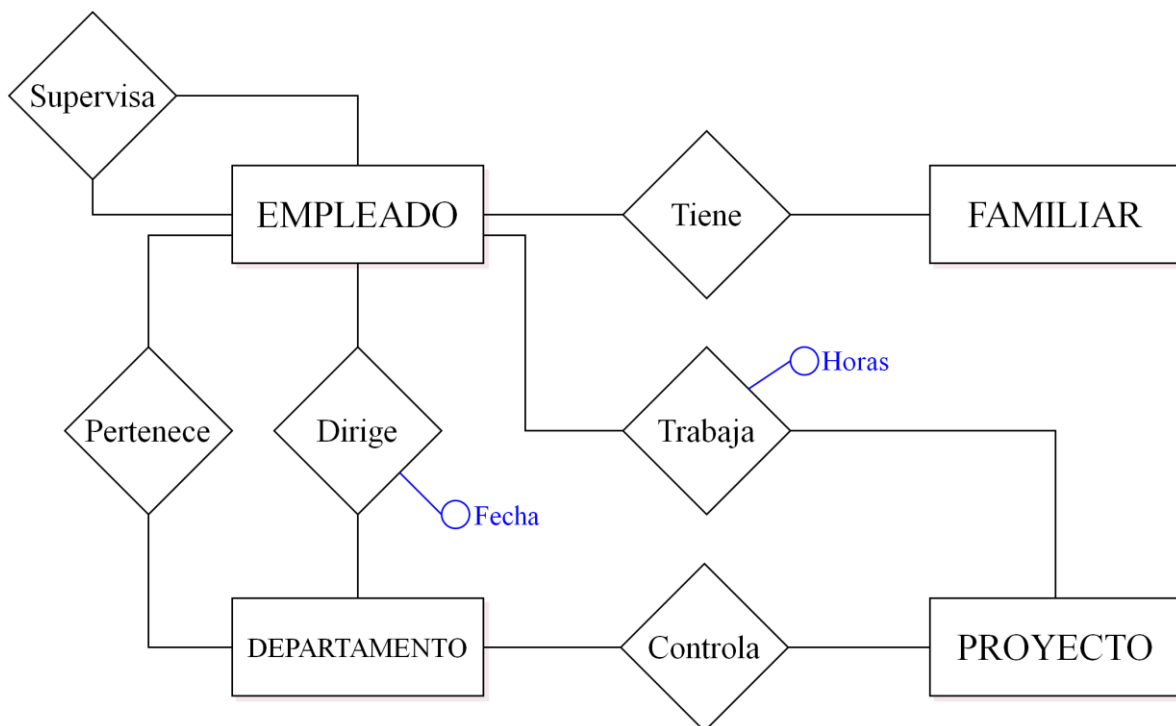
P.e.: Los empleados tienen un supervisor, que también es empleado de la compañía.



2 Entidades + de 1 relación



Todas las Relaciones del ejemplo





4.2.- Atributos de una Relación

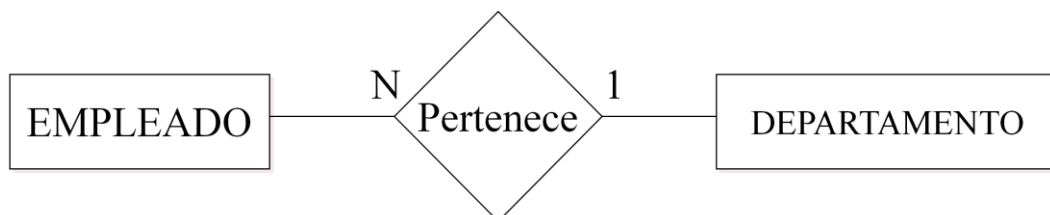
Las relaciones también pueden tener atributos, igual que las entidades.

Por ejemplo, *el nº de horas que un empleado trabaja en un proyecto es un atributo de la relación **trabaja** (no puede ser ni de Empleado ni de Proyecto).*

(En el ejemplo anterior lo hemos marcado en azul)

4.3.- Tipo de Relación

La **CARDINALIDAD** especifica el *número de ocurrencias* de una entidad que pueden intervenir en la relación por cada ocurrencia de la otra entidad.



La relación **PERTENECE** entre **DEPARTAMENTO** y **EMPLEADO** tiene una cardinalidad de **1:N**.

(un departamento relacionado con muchos empleados, pero un empleado con un departamento)

Clases de Relaciones

Las distintas clases de Relaciones que podemos distinguir serian:

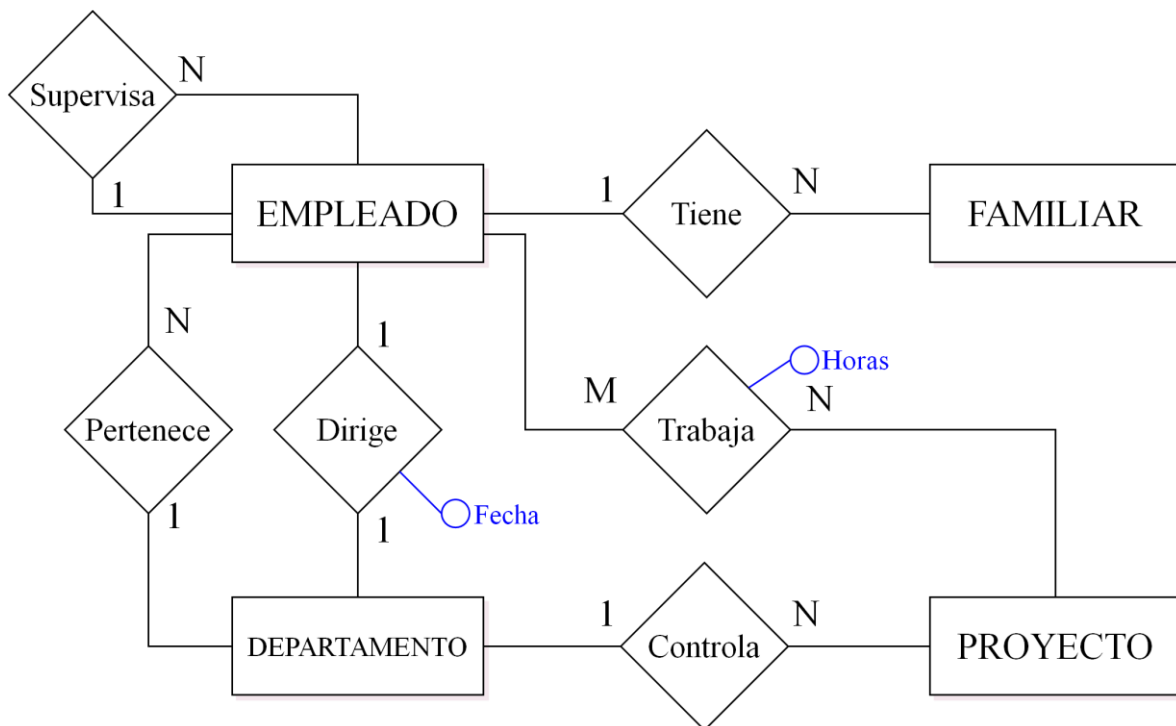
1:1 → Como máximo una ocurrencia de cada.

1:N → En una entidad una ocurrencia y en la otra muchas.

M:N → Hay más de una ocurrencia en cada entidad



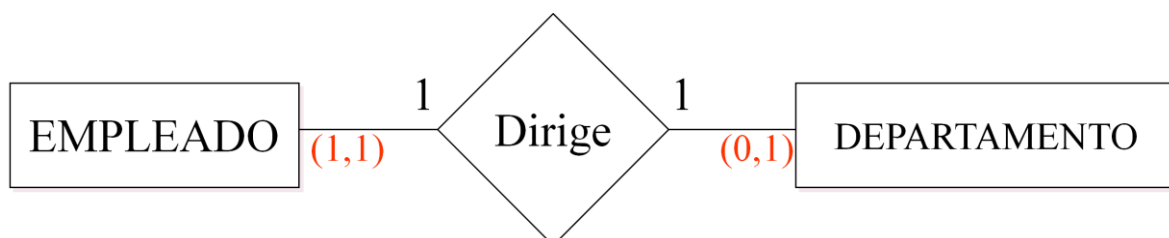
Ejemplo de Clases de Relaciones



5.- Modelo Extendido

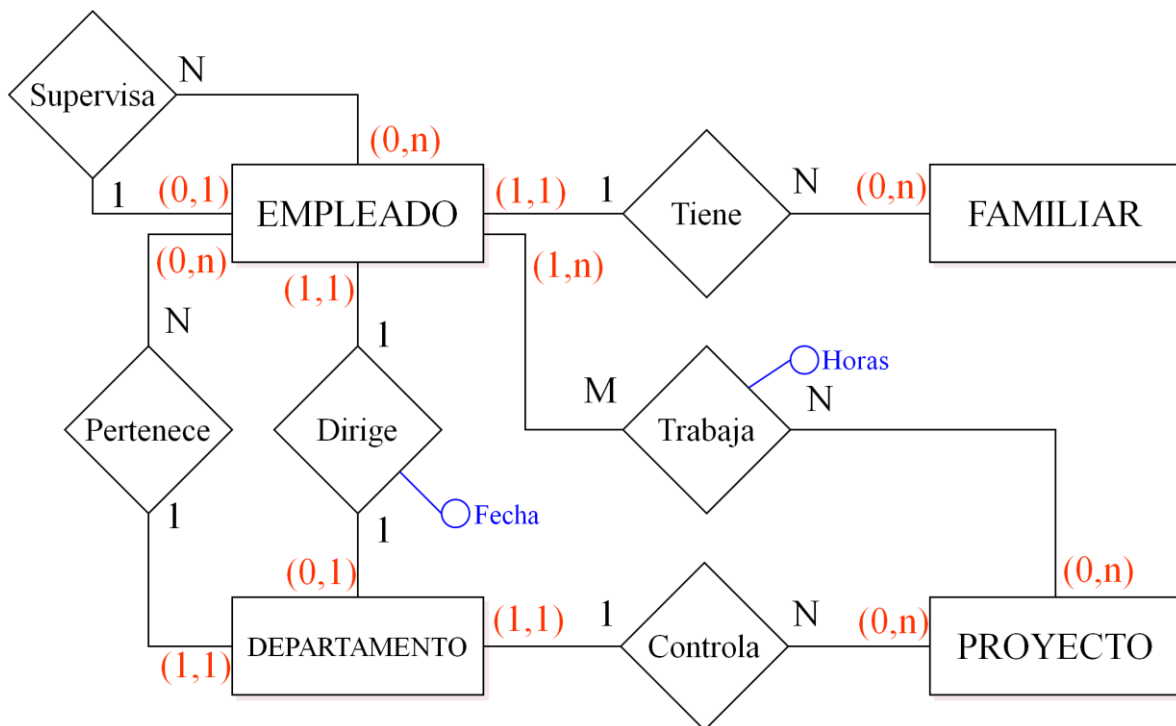
5.1.- Cardinalidad máxima y mínima

Cardinalidad máxima y mínima de una entidad que participa en una relación, son respectivamente *el número máximo y mínimo de ocurrencias de esta entidad que están relacionadas con una ocurrencia de la otra entidad*.





Ejemplo



5.2.- Entidades DÉBILES

No todas las entidades son iguales, en **las normales (REGULARES)** las ocurrencias tienen existencia propia.

En cambio, las **entidades DÉBILES**, la existencia de ocurrencias dependen de la existencia de la ocurrencia de otra entidad.

(Si desaparece esta última, también desaparecen las dependientes)

Por ejemplo, los familiares de Antonio Rico podrían ser (Marta, mujer), (Isabel, hija) y (Pedro, hijo). Si desaparece el empleado Antonio Rico, también deberían de desaparecer sus familiares.



Representación Entidades Débiles

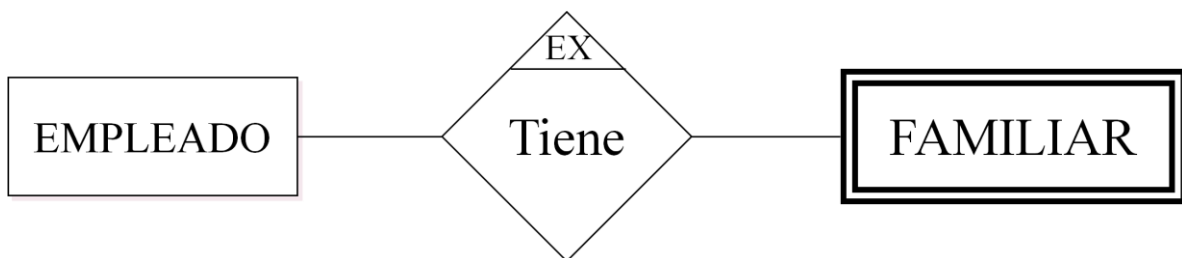
Se representan mediante un doble rectángulo:



La cardinalidad mínima y máxima de la entidad regular en la relación con la débil es siempre de (1,1)

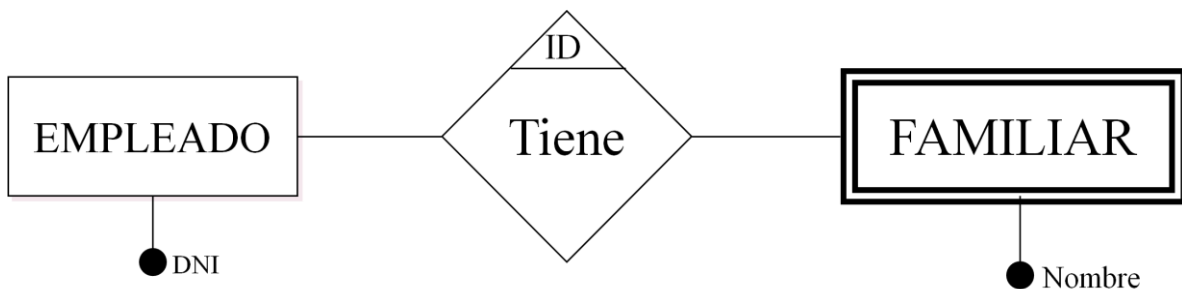
Dependencia de EXISTENCIA

En el ejemplo anterior diremos que la entidad débil tiene una **DEPENDENCIA EN EXISTENCIA**.



Dependencia de IDENTIFICACIÓN

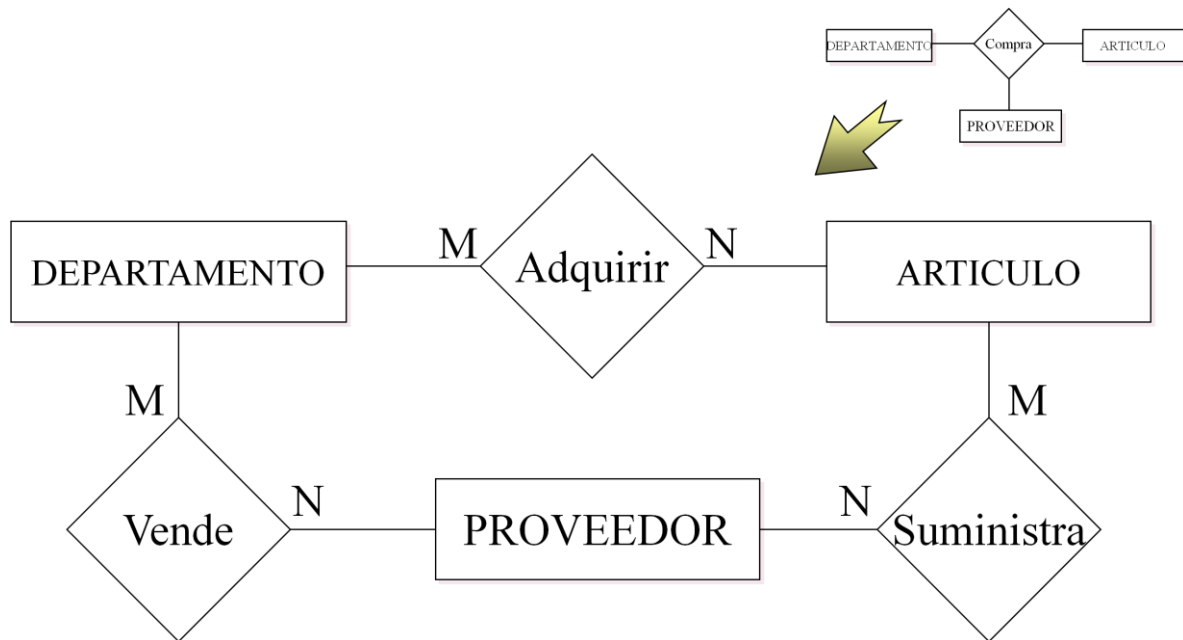
Si además de la dep. de existencia consideramos que para identificar una ocurrencia de la entidad Débil nos hace falta la clave de la entidad regular de la que depende, esta dependencia (todavía más restrictiva) la llamaremos **DEPENDENCIA DE IDENTIFICACIÓN**.





6.- Relaciones de grado mayor que 2

Las relaciones ternarias, debido a su complejidad, se pueden descomponer en tres binarias:



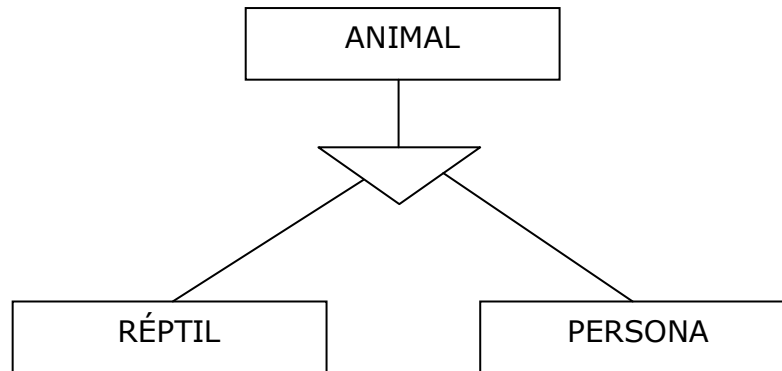
7.- Generalizaciones

Las generalizaciones, nos proporcionan un mecanismo de abstracción que permite especializar una entidad (que se denominará **supertipo**) en **subtipos**, o lo que es igual, generalizar los subtipos en el supertipo. La abstracción correspondiente a este tipo de relación entre entidades se denomina **es_un**. Por ejemplo, una "Persona" es un "Animal" y un "Reptil" es un "Animal"; en este caso, "Animal" puede considerarse el supertipo y "Persona" y "Reptil" son subtipos de "Animal".

Podremos identificar generalizaciones si encontramos una serie de atributos comunes a un conjunto de entidades. Estos *atributos comunes describirán el supertipo y los atributos particulares permanecerán en los subtipos*. Puede ocurrir que los subtipos no tengan atributos propios; en ese caso, sólo existirán subtipos si éstos van a participar en relaciones.



La representación de este tipo de relación, es un triángulo invertido, con la base paralela al rectángulo que representa el supertipo y conectado a este y a los subtipos. Las cardinalidades son siempre (1,1) en el supertipo y (0,1) en los subtipos.





II. Modelo Relacional

1.- ORIGEN Y OBJETIVOS

En 1970 Codd creó el Modelo RELACIONAL, con una base matemática muy sólida (la de las relaciones), donde los datos se estructuran en forma de **relaciones** (tablas).

Fue a partir de los años 80, cuando la tecnología lo permitió, con la salida de mejores productos, como por ejemplo el Oracle (1979). Entonces su implantación ha sido aplastante.

Objetivos del Modelo Relacional:

- **Fidelidad**, para originar esquemas que representen fielmente la información que existe en el dominio del problema.
- **Independencia física**, para la manera de guardar los datos no influya en su manipulación lógica.
- **Independencia lógica**, para que las vistas externas no se vean afectadas por cambios en el esquema conceptual de la B.D.
- **Flexibilidad**, para poder ofrecer los datos a cada usuario de la forma más adecuada a su aplicación.
- **Uniformidad**, las estructuras lógicas de los datos presentan un aspecto simple y uniforme (tablas).
- **Sencillez**, las características anteriores, unidas a unos lenguajes de usuario sencillos, hacen que el Modelo Relacional sea fácil de entender y de utilizar por el usuario final.

2.- ESTRUCTURA DEL MODELO RELACIONAL

El Elemento base del Modelo Relacional es la **RELACIÓN**, que será una tabla o matriz bidimensional con unas características o restricciones.

EMPLEADO			
Dni	Nombre	Sueldo	Fecha_n
18876543	Llopis Bernal, Jaime	200.000	25-2-77
18900111	Garrido Vidal, Rosa	150.000	8-6-60
18944310	Nebot Aliaga, Carmen	300.000	18-5-62



La Relación viene *identificada por un **NOMBRE*** (Empleado).

Las Filas, donde tenemos la información de las ocurrencias, se denominan **TUPLAS**.

Las columnas, que serán las características que nos interesan de los individuos, se llamarán **ATRIBUTOS** (Campos).

El conjunto de posibles valores que puede coger un atributo lo llamaremos **DOMINIO**.

CARDINALIDAD de una relación es el número de tuplas (en el ej. 3).

GRADO de una relación es el número de atributos (ej. 4).

Una **Clave Candidata** es un atributo o conjunto de atributos que identifican unívocamente cada tupla de la relación.

De todas las Claves candidatas, triaremos una que será la **Clave Principal** o clave primaria.

Puede darse el caso de un atributo no coja ningún valor para una determinada tupla, entonces le daremos el valor **NULO**.

3.- RESTRICCIONES

Igual que en otros modelos, existen restricciones, estructuras u ocurrencias no permitidas.

Tipos de restricciones:

- **INHERENTES**: impuestas por el propio modelo.
- **DE USUARIO** (semánticas): en las cuales es el usuario quien prohíbe, porque el modelo se lo permite en determinadas circunstancias.

3.1.- Restricciones **INHERENTES**

Impuestas por el propio modelo. Consideramos las siguientes:

- **Valores atómicos**: cada valor de la tabla ha de ser simple.
- **Tuplas distintas**: no puede haber dos tuplas iguales.
- **Clave Principal**: ha de existir una clave principal o primaria que identifique de forma unívoca las tuplas. (por tanto, no podrá coger valores nulos y tampoco podrá repetirse).



3.2.- Restricciones de Usuario (SEMÁNTICAS)

Condiciones que ponemos para que el esquema de la BD, refleje lo mejor posible la realidad.

(a) Restricción de Dominio:

- El valor de un atributo ha de ser un valor atómico. Definiendo claramente el dominio, no aseguramos que el atributo no puede coger valores incorrectos.
- El **dominio** será de un tipo determinado, eligiendo de una gama extensa: entero corto, entero, entero largo, real, cadena de caracteres,...
- EMPLEADO (DNI: entero(8); Nombre: carácter(30); Dirección: carácter(30); Teléfono: entero(9); Sueldo: entero(6); Fecha_n: fecha)

(b) Restricción de Clave:

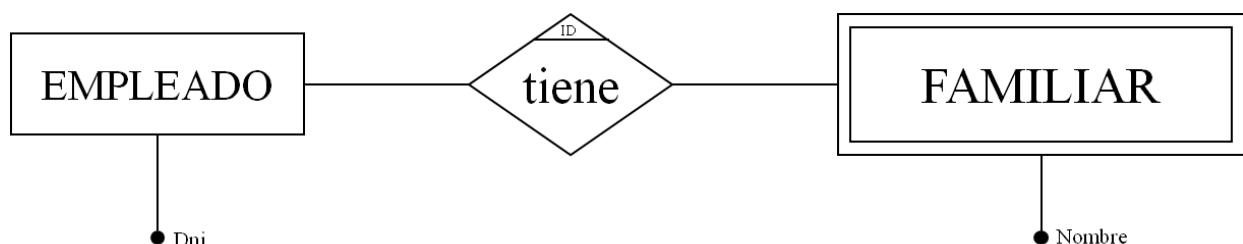
Permite declarar uno o varios atributos como **clave Principal o Primaria** (Primary Key).

"La obligatoriedad de declarar una clave principal era una restricción inherente, pero lo que es una restricción de usuario es la elección de la clave principal, y la consecuencia de que no podrá coger valores nulos ni repetidos". Aunque estas características también las puede tener otros atributos:

- Unicidad (**UNIQUE**): no se puede repetir. (Access ==> Indexado sin duplicados)
- Obligatoriedad (**NOT NULL**): no puede coger un valor nulo. (Access ==> Requerido)

(c) Integridad Referencial:

Veamos un ejemplo:





Si en la tabla Familiar, tenemos un atributo Dni_emp) que es clave (primaria o candidata) de otra tabla (Empleado ==> DNI), todo valor de aquel atributo ha de concordar con un valor de la clave de Empleado (*no podremos poner en familiar un Dni que no tenga ningún empleado en la empresa*). Por lo tanto, Dni_emp es una **Clave Externa** (clave ajena).

Las relaciones no tienen por qué ser distintas, pueden ser la misma. Así, si consideramos el supervisor, este ha de ser de la empresa.

- EMPLEADO (Dni, Nombre, Dirección, Teléfono, Sueldo, Fecha_n, Supervisor)
- Supervisor es una clave externa, pero de la misma tabla (No todos los SGBD permiten una clave externa reflexiva, Access, por ejemplo, no puede hacerlo).

Representación de claves externas:

- EMPLEADO (Dni, Nombre, Dirección, Teléfono, Sueldo, Fecha_n)
 - FAMILIAR (Dni_emp, Nombre, Parentesco)
- 

Esto impedirá que introduzcamos valores no correctos o inexistentes.

¿Qué pasaría si borramos un empleado, o modificamos su Dni? ¿Qué hacemos con sus familiares? Podríamos hacer tres acciones:

- No dejar borrarlo o modificarlo (**NO ACTION**)
- Borrar también los familiares o cambiarlos en cascada (**CASCADE**)
- Cambiar el valor de la clave externa al valor nulo o un valor predeterminado.

(d) Otras restricciones:

Otros SGBD, más potentes, permiten restricciones consistentes en comprobar una determinada condición después de una actualización:

- **Verificación (CHECK)**: si la condición no se cumple después de la actualización, se deshace. Sirve muy bien para definir un dominio, entre otras cosas. Por ejemplo:

CHECK Sueldo >0

CHECK (Año(Fecha_n)<Año(Hoy)) and ((Año(Hoy)-Año(Fecha_n))<65)

- **Disparador (TRIGGER)**: si se cumple la condición se ejecuta un procedimiento definido por el usuario. (Este concepto es más potente, ya que da una respuesta procedimental, donde se puede hacer cualquier cosa).



4.- TRANSFORMACIÓN DEL M E/R AL RELACIONAL.

4.1.- Entidades.

Entidades Normales:

Toda entidad normal se transformará en una tabla, con todos sus atributos, que se consideran como simples. Se tría uno o varios como clave principal, y lo denotaremos subrayándolo.

- EMPLEADO (Dni, Nombre, Dirección, Teléfono, Sueldo, Fecha_n)
- DEPARTAMENTO (Num_d, Nom_d)
- PROYECTO (Num_p, Nom_p)

Entidades Débiles:

Toda entidad débil S depende de otra T, se transformará en una tabla con todos sus atributos, y se añade el o los atributos de la clave principal de T. Este atributo será clave externa de S, marcándolo con un doble subrayado. Si además, la dependencia es en Identificación, la clave externa formará parte de la clave principal.

FAMILIAR (Dni_e, Nom_f, Fecha_n, Parentesco)

4.2.- Relaciones 1:1

Por cada relación 1:1, que afecta a dos entidades S y T, se triará una de ellas, S, donde se incluirá como a clave externa la clave principal de T.



Siempre es conveniente elegir que S participe de forma total, es decir, que todas sus ocurrencias participan en la relación, y por tanto la cardinalidad mínima y máxima sea (1,1) -por cada ocurrencia de S hay como mínimo y como máximo una de T-



Entonces quedaría:

DEPARTAMENTO (Num_d, Nom_d, Director, Fecha)

Es mejor elegir la de participación total porque todos los departamentos tienen director, pero no todos los empleados son directores. (p.e.: si pusiéramos EMPLEADO (Dni, ..., Dep_que_dirige), muchas veces estaría vacío).

4.3.- Relaciones 1:N

Por cada relación 1:N, que no sea una relación débil, entre S y T, donde S participa con un grado de cardinalidad 1, y T con un grado N, se incluye como una clave externa en T la clave principal de S, además de todos los atributos de la relación.

En el ejemplo:

- Para la relación **pertenece** incluiremos el atributo **Departamento a EMPLEADO**.
- Para la relación **controla** incluiremos el atributo **Departamento a PROYECTO**.
- Para la relación **supervisa** incluiremos el atributo **Supervisor a EMPLEADO** (es reflexiva).
- EMPLEADO (Dni, Nom-e, Dirección, Teléfono, Sueldo, Fecha-n, Departamento, Supervisor)
- PROYECTO (Num-p, Nom-p, Departamento)

4.4.- Relaciones M:N

Por cada relación M:N construiremos una nueva tabla, donde se incluirán como una clave externa las claves principales de las dos Entidades, y además su combinación constituirá (o formando parte de ella) la clave principal.

TRABAJA (Dni, Num-p, Horas)

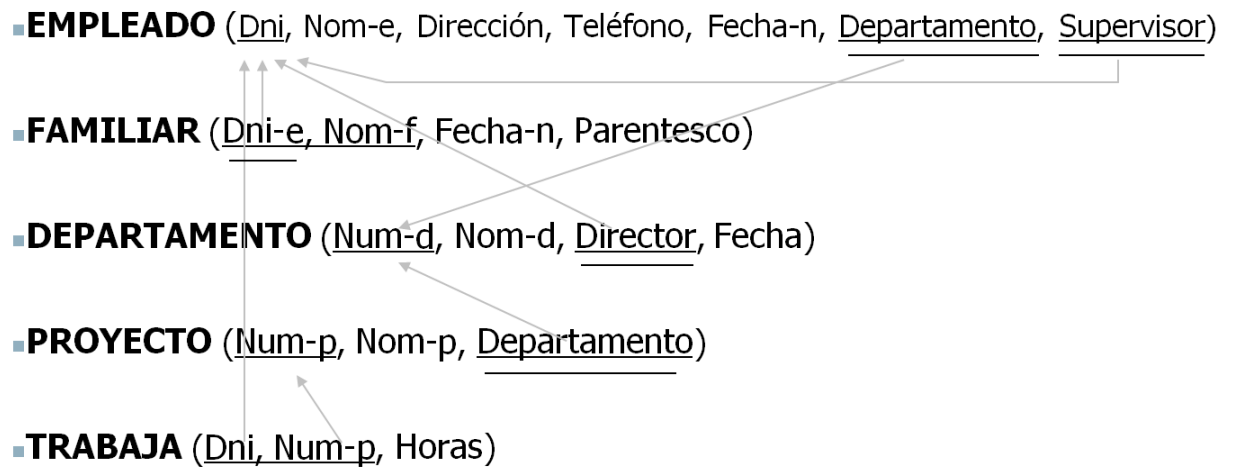
4.5.- Relaciones Ternarias

En una relación ternaria o superior construiremos una nueva tabla, donde incluiremos como clave externas las claves principales de todas las entidades, además de los atributos de la relación.



Ejemplo:

- DEPARTAMENTO (Num-d, ...)
- ARTICULO (Cod-art, ...)
- PROVEEDOR (Cod-pr, ...)
- COMPRA (Num-d, Cod-art, Cod-pr, cantidad)
- Esquema lógico del Ejemplo: EMPRESA



5.- ALGEBRA RELACIONAL

Hasta ahora hemos diseñado una B.D. Relacional, pero para lo que realmente *queremos una B.D. es para consultarla*, y obtener la información que nos interesa, **manipularla**.

Por medio del **ALGEBRA RELACIONAL**, haremos **operaciones sobre las tablas**, combinándolas, seleccionando lo que nos importa,..., en definitiva *manipulándola*.

El *resultado* será una nueva tabla, que será el resultado final o servirá para hacer otra operación. Se dividirán en dos grupos:

- **UNARIAS**: afectan a una sola tabla.
- **BINARIAS**: afectan a dos tablas.



5.1.- Operaciones UNARIAS.

1) Selección (select)

Selecciona un subconjunto de las tuplas, aquellas que cumplen una condición determinada. Lo representaremos:

S (tabla, (condición)) o también $\sigma_{\text{condición}}$ (tabla)

Por ejemplo:

S (EMPLEADO, (Sueldo > 200.000))

S (FAMILIAR, (Parentesco = 'Hijo' o Parentesco = 'Hija'))

Como podemos observar el **grado de la tabla resultante es el mismo, y el número de tuplas menor o igual.**

El caso de encadenar selecciones, es el mismo que poner una condición compuesta:

$S (S (R, (cond1)), (cond2)) \equiv S (R, (cond1 \text{ i } cond2))$

2) Proyección

Selecciona un subconjunto de atributos (columnas), lo representaremos como:

P (tabla, (atributos)) o también $\pi_{\text{atributos}}$ (tabla)

Ejemplo:

P (EMPLEADO, (Nombre, Sueldo))

El grado será menor o igual. Pero el número de tuplas también puede ser menor o igual.

Las operaciones de selección y proyección se pueden encadenar:

$P (S (EMPLADO, (Departamento = 5), (Nombre, Sueldo))$

5.2.- Operaciones BINARIAS

Afectaran a dos tablas. En las dos primeras operaciones, Unión y Diferencia, las dos tablas han de tener los mismos atributos. En la otra, el Producto Cartesiano, no.



1) Unión (R U T)

En el resultado estarán las tuplas de R y las de T (si alguna estuviera en las dos, no se ha de repetir).

Ejemplo:

$P (S(EMPLEADO, (Departamento = 5)), (Dni)) \cup P (S(EMPLEADO, (Departamento=5)), (Supervisor))$

Mientras que los atributos no son iguales, sus dominios son compatibles, y por tanto se puede hacer la unión.

2) Diferencia (R - T)

El resultado contendrá las tuplas que pertenecen a R y no pertenecen a T.

R			T			R-T		
A	B	C	A	B	C	A	B	C
1	0	1	1	1	0	1	0	0
1	0	0	1	0	1	0	3	2
0	3	2	1	2	1	0	2	0
1	2	3	2	0	0			
0	2	0	1	2	3			

3) Producto Cartesiano (R x T)

Aquí no cabe que los atributos sean iguales. Tendremos los atributos de R y de T, y las tuplas serán todas las posibles combinaciones de las tuplas de las dos tablas.

Es decir, *la primera de R combinada con todas las de T, la segunda de R combinada con todas las de T, ...*

Si R tiene **m** filas y T tiene **n**, el producto cartesiano tendrá **m * n**.



Ejemplo:

$S (R \times T , (Num-d = Departamento))$

Entonces tendremos una cosa muy útil, que es la información de los familiares con la del empleado. Esto es una cosa habitual que definiremos una operación que realiza exactamente esto **reunión o join**.

5.3.- OPERACIONES DERIVADAS.

Las operaciones que veremos ahora se pueden deducir de las anteriores.

1) Intersección ($R \cap T$)

Entrarán en el resultado todas la tuplas presentes en las dos relaciones. R y T han de tener los mismos atributos (mejor dicho, atributos de dominios compatibles)

Esta operación es equivalente a: $R \cap T = R - (R - T)$

2) División (R/T)

R actúa como dividendo, y T como divisor. Se ha de cumplir que los atributos de T sean un subconjunto (estricto) de los de R . Llamaremos t a los atributos de T y r a los de R , y se cumple que $t \leq r$. Llamaremos q a los atributos de R que no pertenecen a T , es decir, $q = r - t$.

El resultado de la división tendrá los atributos q , y aparecerá una tupla f_q cuando, por cualquier tupla de T f_t existe una tupla de R f_r que es la combinación de la de T y la del resultado:

$$f_r = f_t + f_q$$

La utilización de la división es muy ocasional, en consultas del estilo del ejemplo (trabajadores que trabajan en todos los proyectos,...)

3) Reunión ($R * T$)

La operación de **REUNIÓN (JOIN)**, también llamada **unión natural**, es de las más importantes del álgebra relacional.



Es el resultado de realizar el producto cartesiano entre las dos tablas, y después aplicar una selección preestablecida que se denomina selección o condición de la reunión.

La condición puede ser cualquiera, pero habitualmente será una igualdad entre dos atributos del mismo nombre (o mismo dominio), y algunos autores la han rebautizado como **EQUIREUNIÓN (EQUIJOIN)**.

La reunión más utilizada será una reunión entre dos tablas con la condición de igualdad entre la clave de una, y la clave externa de otra (que hace de referencia a la primera). En estos casos no cabe que estén presentes los dos atributos que se utilizan para la igualdad, con uno es suficiente. Esta reunión es la REUNIÓN NATURAL.

6.- LENGUAJES RELACIONALES.

El lenguaje más conocido es el **SQL**, que es muy completo, ya que permite definir datos (**DDL**), y también manipularlos (**DML**). Este último se basa en el álgebra relacional, y básicamente constará de sentencias de la forma:

SELECT atributos

FROM tablas

WHERE condición;

El **QBE** (Query By Example) se basa en el cálculo relacional, que es una teoría paralela al álgebra relacional en el cual se definen variables de tipo tabla, se utilizan operadores entre las variables, y también los cuantificadores $\forall$ (para todos) y $\exists$ (existe).

La particularidad de QBE es la sencillez de hacer consultas a los no expertos. Por medio de una plantilla podremos colocar los atributos que queremos visualizar, el orden, los criterios de selección,... (Consultas de Access).



III. Normalización

1.- TEORÍA DE LA NORMALIZACIÓN

El diseño de una base de datos relacional se puede realizar mediante la metodología que acabamos de exponer, aplicando al mundo real, en una primera fase, un modelo semántico como el ME/R, a fin de obtener un esquema conceptual; en una segunda fase, se transforma dicho esquema al modelo relacional mediante las correspondientes reglas de transformación. Si bien nosotros insistimos en las ventajas de este enfoque, existe otra posibilidad que es plasmar directamente en el modelo relacional nuestra percepción del mundo real, obteniendo el esquema relacional sin realizar ese paso intermedio que es el esquema conceptual.

Aunque, en general, la primera aproximación produce un esquema relacional estructurado y con poca redundancia, por lo que no es imprescindible verificar la "bondad" del esquema obtenido, siempre es conveniente aplicar un conjunto de reglas, conocidas como teoría de la normalización, que nos permiten asegurar que un esquema relacional cumple unas ciertas propiedades. En el segundo enfoque, es decir, cuando no se ha aplicado la metodología de diseño anteriormente expuesta, la teoría de normalización resulta imprescindible.

Entre los problemas que puede presentar un esquema relacional cuando el diseño es inadecuado cabe destacar:

- Incapacidad para almacenar ciertos hechos.
- Redundancias y, por tanto, posibilidad de inconsistencias.
- Ambigüedades.
- Pérdida de información (aparición de *tuplas espurias*).
- Pérdida de ciertas restricciones de integridad que dan lugar a interdependencias entre los datos (*dependencias funcionales*).
- Aparición en la base de datos, como consecuencia de las redundancias, de estados que no son válidos en el mundo real; es lo que se llama *anomalías de inserción, borrado y modificación*.

El esquema relacional debe ser, por tanto, analizado para comprobar que no presenta los problemas anteriormente citados, evitando así la pérdida de información y la aparición de inconsistencias.

Veamos un ejemplo de estos problemas derivados de un diseño inadecuado. En la tabla se muestra la relación ESCRIBE que almacena datos sobre los libros (*Cód_libro*, *Título*, *Editorial*, *Año*) y sobre los autores que los han escrito (*Autor* y *Nacionalidad*). Si observamos esta relación, vemos que presenta varios de los problemas enumerados anteriormente.

**ESCRIBE**

AUTOR	NACIONALIDAD	COD_LIBRO	TITULO	EDITORIAL	Año
Date, C.	Norteamericana	23433	Databases	Adisson-W.	1990
Date, C.	Norteamericana	54654	SQL Standard	Adisson-W.	1986
Date, C.	Norteamericana	53235	Guide to Ingres	Adisson-W.	1988
Codd, E.	Norteamericana	97875	Relational M.	Adisson-W.	1990
Gardarin	Francesa	34245	Base de Datos	Paraninfo	1986
Gardarin	Francesa	55366	Comparación BD	Eyrolles	1984
Va1durez	Francesa	86754	Comparación BD	Eyrolles	1984
Kim, W.	Norteamericana	32176	00 Databases	ACM Press	1989
Lochovsky	Canadiense	23456	00 Databases	ACM Press	1989

Los principales problemas de esta relación se derivan de la gran cantidad de redundancia que presenta. Por ejemplo, la nacionalidad del autor se repite por cada libro que ha escrito; y algo análogo sucede, cuando un libro tiene más de un autor, con la editorial y el año de publicación. Esta redundancia produce a su vez:

- *Anomalías de inserción*, ya que dar de alta un libro obliga a insertar en la base de datos tantas tuplas como autores tenga el libro.
- *Anomalías de modificación*, ya que cambiar la editorial de un libro obliga a modificar todas las tuplas que corresponden a ese libro.
- *Anomalías de borrado*, ya que el borrado de un libro obliga a borrar varias tuplas, tantas como autores tenga ese libro y, viceversa, el borrado de un autor nos lleva a borrar tantas tuplas como libros ha escrito ese autor.

Vemos, por tanto, que la actualización (alta, baja o modificación) de un solo libro o de un solo autor nos puede obligar a actualizar más de una tupla, dejándose la integridad de la base de datos en manos del usuario. Al riesgo de la posible pérdida de integridad hay que añadir la falta de eficiencia asociada a estas múltiples actualizaciones.

Además de estas anomalías de inserción, borrado y modificación, existen otros problemas adicionales, como la imposibilidad de almacenar ciertos hechos o la desaparición de información que desearíamos mantener en la base de datos. Por ejemplo, si se quisiera incluir información sobre un autor del que no existiera ningún libro en la base de datos, no sería posible, ya que el atributo `cod_libro` forma parte de la clave primaria de la relación; ni tampoco podríamos introducir obras anónimas (recuerde el lector la regla de integridad de entidad que no permite los nulos en ningún atributo que forme parte de una clave primaria).

Por otro lado, al dar de baja un libro, se pierden también los datos de sus autores (si éstos sólo tuviesen ese libro en la base de datos) y, viceversa, si borramos un autor desaparecen de nuestra base de datos los libros escritos por él (a no ser que el libro tuviera más de un autor).



Esta relación presenta todos estos problemas debido a que atenta contra un principio básico en todo diseño:

"Hechos distintos se deben almacenar en objetos distintos"

en este caso, en relaciones distintas, con lo que se habrían evitado redundancias y, por tanto, los problemas que acabamos de describir.

Si se hubiera llevado a cabo un diseño riguroso no se habría presentado una relación de este tipo. El problema es que a menudo no se llega a comprender completamente o a representar de forma precisa el Universo del Discurso, debido a una excesiva premura al realizar el análisis o a carecer el analista de conocimientos sobre metodologías de diseño de bases de datos o de experiencia para aplicarlas adecuadamente; también el problema deriva muchas veces de una falta de comunicación entre el analista y el usuario.

Si se siguiera la metodología de diseño propuesta, realizando un buen diseño conceptual en el modelo E/R, seguido de una cuidadosa transformación al modelo relacional, se evitarían en gran parte estas anomalías, obteniéndose en general un esquema exento de errores. Sin embargo, ante las posibles dudas respecto a si un determinado esquema relacional es o no correcto, será preferible aplicar siempre a dicho esquema un método formal de análisis que determine lo que pueda estar equivocado en el mismo y nos permita llegar a otro esquema en el que se asegure el cumplimiento de ciertos requisitos; este método formal, como ya hemos indicado, es la **teoría de la normalización**.

La teoría de la normalización evita las redundancias y las anomalías de actualización, obteniendo relaciones más estructuradas que no presenten los problemas que comentábamos anteriormente. Así, en lugar de la relación del ejemplo que aparecía en la figura 8.8, se podría haber diseñado el siguiente esquema relacional:

LIBRO (Cód libro, Título, Editorial, Año)

AUTOR (Nombre, Nacionalidad)

ESCRIBE (Cód libro, Nombre)

donde se ha seguido el principio básico anteriormente enunciado, separando hechos distintos en relaciones distintas, de forma que cada uno de estos esquemas de relación recoge un hecho bien determinado y concreto del mundo real con sus correspondientes atributos; esto es, evidentemente, lo que habría hecho cualquier diseñador medianamente experimentado. Pero ¿cuáles son las razones para haberlo hecho de esta manera?, ¿en qué nos podemos apoyar para afirmar que este diseño es mejor que el anterior?, ¿existen métodos matemáticos y los correspondientes algoritmos que permitan llegar a este diseño?, ¿cómo podríamos conseguir, sin basarnos en la experiencia, que un diseñador advirtiera los errores del primer diseño y, en lugar de la primera relación ESCRIBE, obtuviese el segundo esquema relacional con las tres relaciones LIBRO, AUTOR y ESCRIBE?



La respuesta a estas y a otras preguntas análogas se encuentra en la teoría de la normalización, la cual puede definirse como una *técnica formal para organizar datos*, la cual nos ayuda a determinar qué es lo que está equivocado en un diseño y nos enseña la manera de corregirlo. Por tanto, la teoría de la normalización introduce una formalización en el diseño lógico de bases de datos relacionales, lo que, además, permite mecanizar parte del proceso al poder disponer de instrumentos algorítmicos de ayuda al diseño.

Hemos de advertir que las anomalías a las que da lugar el diseño inadecuado de una base de datos se producen sólo en procesos de actualización y nunca en los de consulta. La aplicación de la teoría de la normalización consigue una disminución de dichas anomalías, evitando muchos de los problemas que se pueden plantear en las actualizaciones. Sin embargo, al mismo tiempo penaliza las consultas al disminuir la eficiencia de las mismas, ya que cuando se aplica el proceso de normalización a una base de datos aumenta el número de relaciones, por lo que una determinada consulta puede llevar consigo el acceso a varias tablas realizando combinaciones entre ellas, lo que, indudablemente, eleva el coste de la consulta (recuérdese que la operación de combinación consume muchos recursos).

2.- NOCIÓN INTUITIVA DE LAS FORMAS NORMALES

La teoría de la normalización se centra en lo que se conoce como formas normales. Se dice que un esquema de relación está en una determinada forma normal si satisface un conjunto específico de restricciones.

La primera forma normal (1FN) fue introducida por Codd en su primer trabajo y, como ya hemos expuesto en el capítulo 5, es una restricción inherente al modelo relacional, por lo que su cumplimiento es obligatorio. **Consiste en la prohibición de que en una relación existan grupos repetitivos, esto es, de que un atributo pueda tomar más de un valor del dominio subyacente.** En realidad, se debe decir que una tabla está normalizada sólo con que se encuentre en 1FN, aunque, a menudo, esta expresión no se aplica con la debida precisión y se afirma que una tabla no está normalizada porque no está en formas normales superiores a la primera.

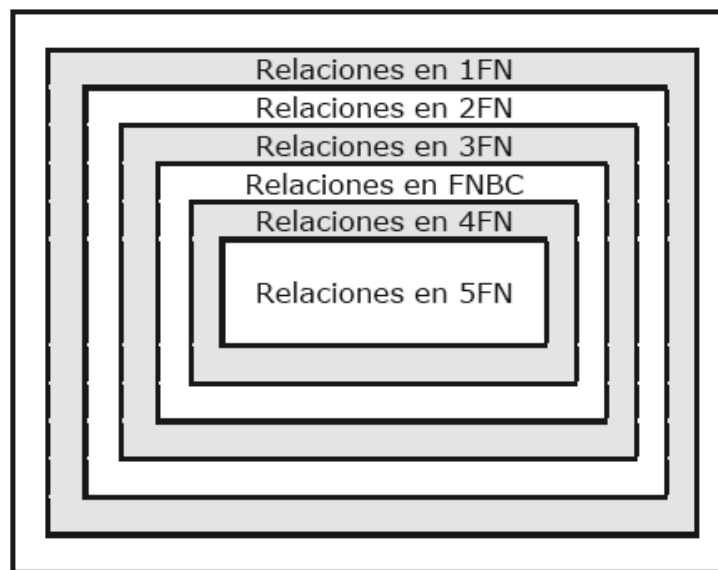
Codd, junto con la 1FN, definió también la *segunda forma normal* (2FN) y la *tercera* (3FN). Posteriormente, otros autores propusieron nuevas formas normales, como la Forma Normal de BOYCE y CODD (FNBC), CODD (1974), y la *cuarta y quinta forma normal* (4FN y 5FN) debidas a FAGIN (1977 y 1979).

Tal como se ilustra en la figura, de todo el universo de relaciones en 1FN, sólo algunas se encuentran en 2FN, y de éstas únicamente una parte está en 3FN, y así sucesivamente; es decir, la 2FN impone más restricciones que la 1FN, la 3FN más que la 2FN, etc., siendo la 5FN la que impone restricciones más fuertes. A fin de evitar las anomalías



ías que describíamos anteriormente es preferible la 5FN, después la 4FN, la 3FN, la 2FN y, por último, la 1FN.

La teoría de la normalización se basa en ciertas restricciones definidas sobre los atributos de una relación, las cuales son conocidas con el nombre de dependencias. Existen varios tipos de dependencias, encontrándose relacionadas la 2ª, 3ª y FNBC con las dependencias funcionales, la 4ª con las dependencias *multivaluadas* y la 5ª con las dependencias de *proyección-combinación*.



Dado que la exposición de la teoría formal de la normalización exige una cierta base matemática, creemos que mostrar intuitivamente las ideas que subyacen en dicha teoría puede ayudar a nuestros lectores a comprender unos principios de diseño aplicables, no sólo al caso del modelo relacional, sino también a otros modelos o, incluso, a sistemas de ficheros. Indudablemente, estas ideas intuitivas no tendrán la precisión ni la formalización de la teoría matemática de la normalización, pero son muchas veces suficientes en la práctica.

Dentro de este enfoque intuitivo podemos decir que **un esquema de relación se encuentra en 2FN si, además de estar en 1FN, todos los atributos que no forman parte de ninguna clave candidata suministran información acerca de la clave completa, no de una parte de la clave.**

Ésta es la causa por la cual la relación ESCRIBE presenta tantas anomalías, y es también el origen de los mismos problemas en la relación PRÉSTAMO del ejemplo que exponemos a continuación. Sea el esquema de relación:

PRÉSTAMO (Núm_socio, Dni_socio, Cód_libro, Fecha_Préstamo, Editorial, País)



donde las claves candidatas son:

(Núm_socio, Cód_libro)

(Dni_socio, Cód_libro)

Se puede observar que ciertos atributos que no forman parte de ninguna clave candidata, como Editorial, nos dan información acerca del libro pero no tienen nada que ver con el socio, por lo que no constituyen una información acerca de la clave completa sino únicamente de una parte de la clave, luego la relación PRÉSTAMO no se encuentra en 2FN.

La solución a los problemas ya expuestos que presenta esta relación es descomponerla en dos:

PRÉSTAMO1 (*Núm_socio, Dni_socio, Cód_libro, Fecha_préstamo*)

LIBROS (*Cód_libro, Editorial, País*)

en el primer esquema existen dos claves (*Núm_socio, Cód_libro* y *DNI_socio, Cód_libro*), y el único atributo (*Fecha_préstamo*) que no forma parte de ninguna clave suministra Información acerca de la totalidad de ambas claves candidatas; por otra parte, en LIBROS la clave es *Cód_libro*, y los dos atributos que no son clave también suministran información acerca de la clave completa, ya que en este caso es simple. Por tanto, ambas relaciones se encuentran en 2FN.

De lo que acabamos de exponer se deduce que toda relación cuya clave está formada por un único atributo está, al menos, en 2FN.

Una relación se encuentra en 3FN si, además de las dos restricciones anteriores (1FN y 2FN), se cumple que los atributos que no forman parte de ninguna clave candidata facilitan información sólo acerca de la(s) clave(s) y no acerca de otros atributos.

Así, por ejemplo, en PRÉSTAMO1 el atributo *Fecha_préstamo* sólo facilita información acerca de las claves, ya que no existe ningún otro atributo no clave, por lo que dicha relación está en 3FN. Sin embargo, en la relación LIBROS el atributo *País* facilita información acerca de la *Editorial*, no encontrándose por tanto esta relación en 3FN.

Para resolver los problemas (redundancias, inconsistentes, etc.) que presenta esta relación, conviene descomponerla en:

LIBROS1 (*Cód_libro, Editorial*)

EDITORIALES (*Editorial, País*)

que están en 3FN, ya que todo atributo no clave sólo facilita información acerca de la clave.



En resumen, podemos decir que, para que una relación se encuentre en 3FN, ha de estar en 1FN y, además, todos sus atributos que no forman parte de ninguna clave candidata "deben ser información referida a la clave, la clave completa y nada más que la clave", KENT (1983).

Al descomponer la relación de partida (que no estaba en 2FN) en tres relaciones que están en 3FN, hemos evitado muchas de las anomalías anteriormente expuestas. Sin embargo, todavía pueden existir relaciones (como PRÉSTAMO1) que, a pesar de encontrarse en 3FN, continúan presentando problemas; para evitarlos, se definieron el resto de las formas normales en las que no vamos a entrar en esta obra.

3.- DEPENDENCIAS FUNCIONALES

Ya hemos señalado que la teoría de la normalización se basa en el concepto de dependencias, hasta el punto de que se la conoce también como *teoría de las dependencias*.

Las dependencias son propiedades inherentes al contenido semántico de los datos que se han de cumplir para cualquier extensión del esquema de relación y forman parte de las restricciones de usuario del modelo relacional. La existencia de una dependencia no se puede demostrar, pero sí afirmar por observación del mundo real que se trata de modelar; por tanto, del análisis de una extensión válida de un esquema de relación nunca podremos deducir la existencia de una dependencia, pero sí podremos, en cambio, llegar a la conclusión de que no existe una determinada dependencia. Por otra parte, si conocemos que una dependencia es cierta para un esquema de relación, podremos asegurar que una extensión de dicho esquema no es válida si no la cumple.

Las dependencias nos muestran algunas importantes interrelaciones existentes entre los atributos del mundo real, cuya semántica tratamos de incorporar a nuestra base de datos; son, por tanto, invariantes en el tiempo, siempre que no cambie el mundo real del cual proceden

Las dependencias funcionales son un tipo especial de dependencias, el más extendido en la práctica, en el cual se basan las primeras formas normales. A continuación vamos a definir dicho concepto, procurando simplificar al máximo el formalismo matemático a él asociado.

Sea el esquema de relación R definido sobre el conjunto de atributos A y sean X e Y subconjuntos de A llamados descriptores. Se dice que Y *depende funcionalmente* de X, o lo que es igual, que **X determina o implica a Y si, y sólo si, cada valor de X tiene asociado en todo momento un único valor de Y**. Representamos esta dependencia funcional de la siguiente forma:

$$X \twoheadrightarrow Y$$



Llamamos *determinante* o **implicante** al descriptor que se encuentra a la izquierda del símbolo de implicación, e **implicado** al descriptor que se encuentra a la derecha.

Sea, por ejemplo, la relación:

LIBRO (*Cód_libro*, *Título*, *Idioma...*)

podemos decir que el código de un libro determina el título del mismo:

Cód_libro --> *Título*

El código del libro (*Cód_libro*) es el implicante (o determinante) en la anterior dependencia y *Título* es el implicado. Esta dependencia también nos dice que el título es una información acerca del libro, ya que una dependencia funcional se puede interpretar diciendo que el implicado es un hecho (en realidad una información) acerca del implicante.

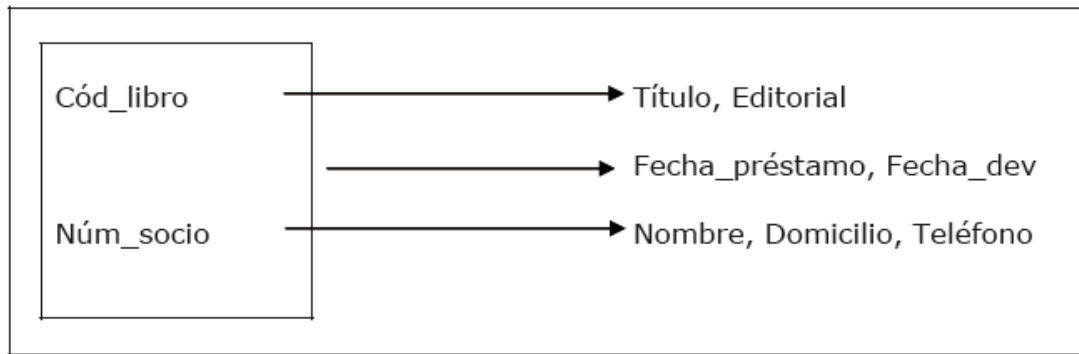
Queremos advertir que la afirmación de que *Cód_libro* determina *Título* no significa que conocido el código de un libro podamos deducir, a partir de él, cuál es su título, a no ser que tengamos una extensión $r(R)$ del esquema de relación que contiene la correspondiente dependencia funcional. Es decir, si para un esquema R tenemos la dependencia funcional:

$X \rightarrow Y$

dado el valor de X no podemos, en general, conocer el valor de Y; solamente nos limitaremos a afirmar que para dos tuplas de cualquier extensión $r(R)$ que tengan el mismo valor de X , el valor de Y será también igual en ambas.

Una herramienta muy útil a la hora de explicitar las dependencias funcionales es el grafo o diagrama de dependencias funcionales, mediante el cual se representa un conjunto de atributos y las dependencias funcionales existentes entre ellos. En el grafo aparecen los nombres de los atributos unidos por flechas, las cuales indican las dependencias funcionales y parten del implicante hacia el implicado. Cuando el implicante de una dependencia no es un único atributo, es decir, se trata de un implicante compuesto, los atributos que lo componen se encierran en un recuadro y la flecha parte de éste, no de cada atributo.

En la figura se presenta un ejemplo de cómo se visualizan las dependencias; podemos observar que *Cód_libro* determina funcionalmente el *Título* del libro y la *Editorial*, como indica la correspondiente flecha; de forma análoga, *Núm_socio* determina el *Nombre*, *Domicilio* y *Teléfono* del socio (se supone que sólo nos interesa un teléfono); mientras que ambos atributos en conjunto *Cód_libro* y *Núm_socio* (lo que se indica haciendo que la flecha parta del recuadro que los incluye) determinan *Fecha_préstamo* y *Fecha_dev.*



Existen otros conceptos, como el de dependencia plena o completa y el de dependencia transitiva, fundamentales en la teoría de la normalización, que veremos a continuación.

3.1.- Dependencia funcional plena o completa

Sea un descriptor compuesto X:

X (X1, X2)

se dice que **Y tiene dependencia funcional completa o plena de X si depende funcionalmente de X pero no depende de ningún subconjunto del mismo**, esto es:

$X \twoheadrightarrow Y$

$X1 \not\rightarrow Y$

$X2 \not\rightarrow Y$

lo que se representa por:

$X \Rightarrow Y$

Supongamos la relación:

PRESTA (Cód_libro, Título, Editorial, Núm_socio, Nombre, Domicilio, Teléfono, Fecha_préstamo, Fecha_dev)

cuyas dependencias funcionales aparecen en la figura anterior, la dependencia funcional:

$Cód_libro, Núm_socio \twoheadrightarrow Fecha_préstamo$

indica que, dado un determinado código de libro y un número de socio, existe una única fecha de préstamo (para simplificar se ha supuesto que un mismo libro no se presta a un mismo socio en varias ocasiones). Ni código de libro, ni tampoco número de socio,



implican, por sí solos, la fecha de préstamo, ya que tanto un libro se puede prestar en varias fechas como un socio puede recibir libros prestados en varias fechas.

Por tanto, la dependencia funcional anterior es completa, lo que se representa:

$Cód_libro, Núm_socio \Rightarrow Fecha_préstamo$

ya que:

$Cód_libro \not\rightarrow Fecha_préstamo$

$Núm_socio \not\rightarrow Fecha_préstamo$

lo que, intuitivamente, se puede interpretar como que *Fecha_préstamo* constituye una información sobre el conjunto de libro y socio, pero esta información no atañe a un libro o a un socio por separado.

Por el contrario, la dependencia:

$Cód_libro, Núm_socio \twoheadrightarrow Editorial$

no es plena, ya que:

$Cód_libro \twoheadrightarrow Editorial$

se dice que, en esa dependencia, *Núm_socio* es un atributo **redundante** o **ajeno** a la dependencia (también llamado extraño).

3.2.- Dependencia funcional transitiva

Sea la relación

$R(X, Y, Z)$

en la que existen las siguientes dependencias funcionales:

$X \twoheadrightarrow Y$

$Y \twoheadrightarrow Z$

y $Y \rightarrow X$

se dice entonces que *Z* tiene una **dependencia transitiva** respecto de *X* a través de *Y*, lo que se representa por:

$X \twoheadrightarrow\twoheadrightarrow Z$

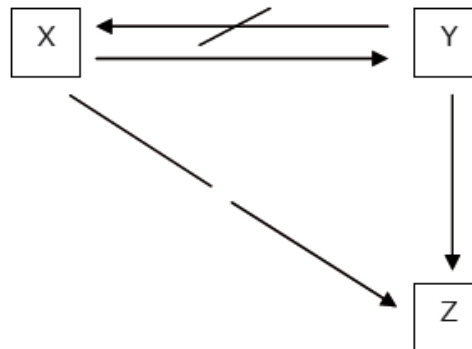
Si además, $Z \rightarrow Y$, se dice que la dependencia transitiva es **estricta**.



En la siguiente figura se presenta un diagrama de dependencias funcionales en el que la dependencia

$X \twoheadrightarrow Z$

es transitiva.



Si consideramos la relación

LIBROS (*Cód_libro*, *Editorial*, *País*)

en donde tenemos, para cada libro, su código, la editorial que lo publica y el país de la editorial (suponemos que una editorial tiene su sede en un único país), se tendrán las siguientes dependencias:

$Cód_libro \twoheadrightarrow Editorial$

$Editorial \twoheadrightarrow País$

$Cód_libro \twoheadrightarrow País$

además, $Editorial \twoheadrightarrow Cód_libro$ (ya que una editorial puede publicar varios libros), la dependencia funcional entre *Cód_libro* y *País* es una dependencia transitiva a través de *Editorial*, representándola por:

$Cód_libro \twoheadrightarrow País$

(lo que se puede interpretar intuitivamente como que *País* es una información sobre el libro, pero **indirectamente**, ya que constituye una información sobre la editorial y ésta, a su vez, sobre el libro).

Además, como $País \twoheadrightarrow Editorial$, la dependencia transitiva es estricta.

En cambio, si tuviéramos la relación:

LIBRO (*Cód_libro*, *Título*, *Editorial*)



con las siguientes dependencias:

Cód_libro --> Título

Título --> Cód_libro

Cód_libro --> Editorial

Título --> Editorial

ninguna de las dependencias *Cód_libro --> Editorial* y *Título --> Editorial* son transitivas, al ser los atributos *Cód_libro* y *Título* equivalentes (ambos se implican mutuamente).

4.- DEFINICIÓN FORMAL DE LAS TRES PRIMERAS FORMAS NORMALES

Expuesto ya el concepto de dependencia funcional, podemos formalizar la definición de segunda y tercera forma normal apoyándonos en la dependencia funcional plena y transitiva.

Primera forma normal (1FN). Recordemos que para que una tabla pueda ser considerada una relación no debe admitir grupos repetitivos, esto es, debe estar en primera forma normal. En el ejemplo de la figura se observa, en la parte a), grupos repetitivos en aquellos libros que tienen más de un autor; para pasar esta tabla (no es una relación puesto que no está en 1FN) a 1FN, habrá que repetir el resto de atributos de la tupla para cada uno de los valores del grupo repetitivo, tal como aparece en la parte b) de la figura. Se dice que una relación está en 1FN si cada atributo toma un único valor del dominio subyacente.

LIBRO (*Código, Título, Autor*)

CODIGO	TÍTULO	AUTOR
02145989	Modelado de datos	Antonio Rico Juan Pérez
86454353	Una guía de DB2	Jimenez Albert
65465465	Bases de datos	María Gómez Andrés Rico

No esta en 1FN
Hay grupos repetitivos





LIBRO (*Código, Título, Autor*)

CODIGO	TÍTULO	AUTOR
02145989	Modelado de datos	Antonio Rico
02145989	Modelado de datos	Juan Pérez
86454353	Una guía de DB2	Jimenez Albert
65465465	Bases de datos	María Gómez
65465465	Bases de datos	Andrés Rico

Esta en 1FN

Segunda forma normal (2FN). Se dice que una relación está en 2FN si:

- Está en 1FN.
- Cada atributo no principal tiene dependencia funcional completa respecto de cada una de las claves.

Por ejemplo, si tenemos la relación: PRESTA (*Cód_libro, Núm_socio, Editorial*), en la que existe la siguiente dependencia funcional:

Cód_libro --> Editorial

y cuya clave está constituida por el descriptor *Cód_libro, Núm_socio*, esta relación no se encuentra en 2FN al venir la editorial determinada sólo por el código de libro y no por la clave completa.

Tercera forma normal (3FN). Se dice que una relación está en 3FN si:

- Está en 2FN.
- No existe ningún atributo no principal que dependa transitivamente de alguna de las claves de relación.

Sea, por ejemplo, la relación:

SOCIO (Núm_socio, Ciudad, País)

que presenta las siguientes dependencias funcionales:

Núm_socio --> Ciudad

Ciudad --> País

Y cuya clave es, evidentemente, *Núm_socio*. Dicha relación no se encuentra en 3FN, al depender *País* transitivamente de la clave a través de *Ciudad*.

Tanto la relación PRESTA como la relación SOCIO presentan los problemas citados anteriormente. Si estas relaciones se llevasen a formas normales más avanzadas se eliminarían, o al menos se atenuarían, estos problemas.



5.- DESCOMPOSICIÓN DE RELACIONES

La transformación de una relación que se encuentra en una determinada forma normal, en otra relación cuya forma normal es superior se realiza por medio del operador de proyección del álgebra relacional.

Sea, por ejemplo, la relación:

PRESTA (Cód_libro, Núm._socio, Editorial)

que, como hemos visto, se encuentra sólo en 1FN (ya que su único atributo no principal, *Editorial*, no depende totalmente de la clave). Si quisiéramos llevar esta relación a una forma normal más avanzada, sería preciso descomponerla, mediante proyecciones, obteniendo varias relaciones. Así, proyectando sobre *Cód_libro*, *Núm_socio* y sobre *Cód_libro*, *Editorial*, tendríamos:

PRESTA1 (Cód_libro, Núm_socio)

PRESTA2 (Cód_libro, Editorial)

Ambas relaciones están en 3FN y han desaparecido las redundancias y las inconsistencias a las que antes aludíamos.

Además, la combinación natural de PRESTA1 * PRESTA2 (por el atributo común *Cód_libro*) nos devuelve la relación original.

De manera análoga, la relación:

SOCIO (Núm_socio, Ciudad, País)

que se encuentra en 2FN, pero no en tercera, podría descomponerse en dos proyecciones:

SOCIO1 (Núm_socio, Ciudad)

CIUDAD (Ciudad, País)

También, al igual que en el caso anterior:

SOCIO1 * CIUDAD = SOCIO

Ciudad

En el proceso de descomposición de relaciones, para llevarlas a formas normales más avanzadas es preciso cumplir unas determinadas reglas:



a.- Descomposición sin pérdida de información

Se dice que una descomposición se ha realizado sin pérdida de información cuando la combinación natural de las proyecciones resultantes nos devuelve la relación original.

Sea la relación:

LIBRO (Cód_libro, Editorial, País)

en la cual tienen lugar las siguientes dependencias funcionales:

Cód_libro --> *Editorial*

Editorial --> *País*

una extensión de esta relación aparece en la parte a) de la figura.

Libro

COD_LIBRO	EDITORIAL	PAIS
654654	Ra-ma	España
665465	Ra-ma	España
876545	Paraninfo	España
987456	Anaya	España
965842	Addison-w.	EE. UU.

Libro1

COD_LIBRO	PAIS
654654	España
665465	España
876545	España
987456	España
965842	EE. UU.

Editorial1

EDITORIAL	PAIS
Ra-ma	España
Ra-ma	España
Paraninfo	España
Anaya	España
Addison-w.	EE. UU.

Libro1 * Editorial 1

COD_LIBRO	EDITORIAL	PAIS
654654	Ra-ma	España
654654	Paraninfo	España
654654	Anaya	España
665465	Ra-ma	España
665465	Paraninfo	España
665465	Anaya	España
876545	Ra-ma	España
876545	Paraninfo	España
876545	Anaya	España
987456	Ra-ma	España
987456	Paraninfo	España
987456	Anaya	España
965842	Addison-w.	EE. UU.

→
Tuplas Espúrias



Supongamos que descomponemos esta relación en:

LIBRO1 (Cód_libro, País)

EDITORIAL1 (Editorial, País)

cuyas extensiones aparecen en la parte b) de la figura. La combinación de estas dos relaciones (ver parte c) de la figura) da lugar a la aparición de *tuplas espurias* que no se encontraban en la extensión original. Se dice que la descomposición de LIBRO ha dado lugar a pérdida de información.

Si en lugar de esta descomposición hubiéramos obtenido:

LIBRO2 (Cód_libro, País)

EDITORIAL2 (Cód_libro, Editorial)

es fácil comprobar que la combinación natural

LIBRO2 * EDITORIAL2

Cód_libro

daría la relación original (sin tuplas espurias).

La condición necesaria y suficiente para que una descomposición se produzca sin pérdida de información es que el atributo común de las dos relaciones sea clave, al menos, en una de ellas; condición que no se cumple en la primera descomposición, pero sí en la segunda.

b.- Descomposición sin pérdida de dependencia funcional

Las dependencias funcionales recogen la semántica del mundo real, por lo que es conveniente conservarlas en el proceso de descomposición.

Sea la misma relación LIBRO del ejemplo anterior que hemos descompuesto en LIBRO2 y EDITORIAL2, donde no ha habido pérdida de información, dado que el atributo común (Cód_libro) es clave en ambas relaciones.

Sin embargo, en esta descomposición se ha perdido una dependencia funcional, ya que en la relación LIBRO2 la única dependencia funcional es:

Cód_libro --> País

y en la EDITORIAL2, la única dependencia es:

Cód_libro --> Editorial



luego la dependencia *Editorial* --> *País* se ha perdido, y con ella también ha desaparecido en nuestro esquema parte de la semántica del mundo real, que nos dice que, dada una editorial, ésta se encuentra en un único país.

c.- Descomposición en proyecciones independientes

La descomposición de una relación *R* en un conjunto de relaciones $\{R_i\}$ se dice que se ha realizado en proyecciones independientes si no ha habido pérdida de información ni de dependencias funcionales.

Si la relación LIBRO de nuestro ejemplo la hubiésemos descompuesto en:

LIBR03 (Cód_libro, *Editorial*)

EDITORIAL3 (*Editorial*, País)

estas dos proyecciones serían independientes, ya que el atributo común es clave de una de las relaciones (la segunda), por lo que no hay pérdida de información; además, tampoco hay pérdida de dependencias funcionales, dado que en LIBR03 tenemos la dependencia:

Cód_libro --> *Editorial*

y en EDITORIAL3, tendríamos:

Editorial --> *País*

Se trata de la mejor descomposición; las relaciones resultantes son equivalentes a la relación original y, en ellas, se han eliminado las anomalías de inserción, borrado y modificación (dado que sólo se ha llegado a 3FN no se puede asegurar que, en todos los casos, se eliminen por completo las anomalías).

Se puede demostrar que es posible descomponer cualquier relación, llevándola a 3FN, sin pérdida de información ni de dependencias funcionales (cosa que no siempre ocurre si se quisiese llevarla a formas normales más avanzadas).

A veces, el proceso de normalización se aplica a la denominada *relación universal*, constituida por el conjunto de atributos de universo del discurso que se desea modelar y por el conjunto de sus dependencias funcionales. Además del método de descomposición, también llamado *análisis*, cuyos fundamentos acabamos de exponer, existe otro método para normalizar una relación que es el método de *síntesis* (en el cual no se entra en esta obra).

Existen procedimientos algorítmicos que realizan la descomposición de relaciones en proyecciones independientes.



6.- CONSIDERACIONES FINALES SOBRE LA TEORÍA DE LA NORMALIZACIÓN

La teoría de la normalización nos ayuda a estructurar mejor las relaciones, evitando posibles redundancias y anomalías, y a representar mejor nuestro mundo real en un esquema relacional.

Sin embargo, no hay que olvidar que al descomponer una relación penalizamos las consultas, provocando una pérdida de eficiencia en las mismas. Aunque, en general, se aconseja llevar los esquemas relacionales al menos a 3FN, existen ciertos casos en los que, una vez realizada la descomposición, exigencias de eficiencia muy estrictas obligan a llevar a cabo el proceso inverso, es decir, una *desnormalización*, combinando las relaciones hasta dejarlas en formas normales anteriores. También en relaciones muy estables, donde apenas se producen actualizaciones (este es, por ejemplo, el caso de ciertas investigaciones estadísticas), puede no ser conveniente avanzar en la normalización.

Por otra parte, si seguimos la metodología de diseño expuesta en la primera parte de este capítulo, obteniendo primero un esquema en el modelo entidad/interrelación y transformándolo después al modelo relacional, el esquema relacional resultante, siempre que todo el proceso se haya realizado correctamente, estará en 3FN (e incluso en formas normales más avanzadas). En este caso, la teoría de la normalización nos servirá para comprobar que el diseño ha sido correcto y, si no lo fuese, podremos aplicar la descomposición para corregir los errores que hubieran podido producirse.